Big Data Application Pipeline: Real-Time Stream Monitor

Activity Report

Name	Your Name
Course / Section	Course code - Big Data / Section
Date	October 2026
Repository	https://github.com/<your-username>/big-data-streamlit-template

1. Objective

To design and implement an end-to-end big-data streaming application that ingests live data from a public Application Programming Interface (API), moves it through a **MongoDB-Kafka-Spark** pipeline, and presents it in a live dashboard (refreshing every 15 seconds) together with a complete history, data aggregation and formatted data export.

2. API Used

The producer polls free, key-less public APIs. Two are enabled by default and two more can be switched on with a single setting (`DATA_SOURCE`) because every API is wrapped by the same adapter interface.

API	Endpoint	Data collected	Notes
CoinGecko (default)	api.coingecko.com/api/v3/simple/price	USD & PHP price and 24 h change for Bitcoin, Ethereum, Solana, Dogecoin, XRP	No key; free tier is rate-limited; prices refresh roughly every 30-60 s
wheretheiss.at (default)	api.wheretheiss.at/v1/satellites/25544	ISS latitude, longitude, altitude (km), velocity (km/h)	No key; ~1 request/s limit; changes every second
Open-Meteo Air Quality (optional)	air-quality-api.open-meteo.com/v1/air-quality	PM2.5, PM10, US AQI, O3, NO2, CO for 5 Philippine cities	No key; hourly model data
USGS Earthquake (optional)	earthquake.usgs.gov/fdsnws/event/1/query	Magnitude and depth of earthquakes around the Philippines	No key; event-driven, de-duplicated by event id

Every API response is normalised into one schema: `timestamp` (ISO-8601 UTC), `value` (numeric), `metric_type`, `sensor_id`, plus `record_id`, `source`, `unit`, `location`, `latitude/longitude`. This lets the downstream Spark job and dashboard stay generic - changing the API never requires changing them.

3. Pipeline Implemented: MongoDB - Kafka - Spark

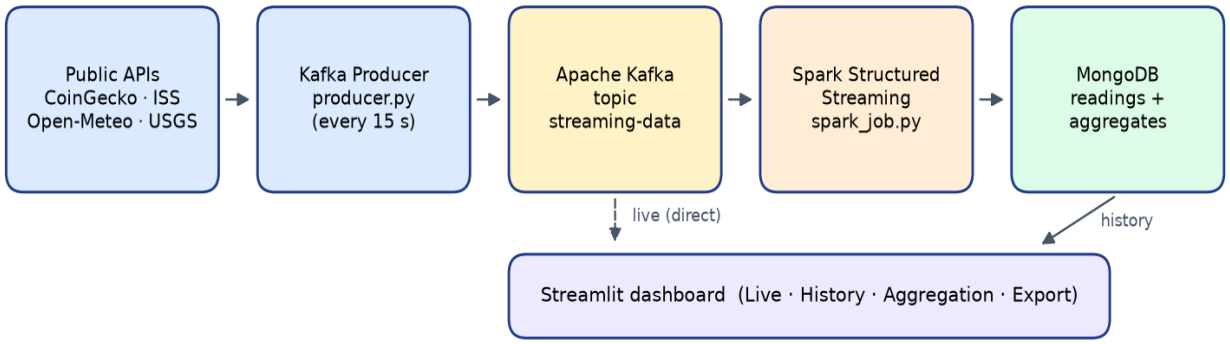


Figure 1. System architecture.

Stage	Technology / file	Role
Ingest	Python producer (producer.py, sources.py)	Polls the API every 15 s with retry/back-off; a failing API never stops the others; publishes JSON keyed by sensor_id.
Buffer	Apache Kafka (topic streaming-data)	Durable, ordered, decoupled message log; lets the dashboard and Spark consume independently.
Process	Spark Structured Streaming (spark_job.py)	Parses and validates JSON, drops malformed records, writes each valid record to MongoDB, and computes 1-minute tumbling-window aggregates (avg, min, max, std, count) with a 2-minute watermark.
Store	MongoDB (storage.py)	Collections *readings* (raw history) and *aggregates* (Spark windows). Upserts keyed by record_id make replays idempotent; compound indexes speed time-range queries.
Serve	Streamlit + Plotly (app.py)	Live view reads Kafka directly; History and Aggregation views query MongoDB; exports via exporters.py.

Why this combination? Kafka absorbs bursts and decouples producers from consumers; Spark gives scalable, fault-tolerant stream processing with event-time windows and checkpointing; MongoDB's flexible documents and time indexes suit semi-structured sensor/API data. Together they scale from this single-machine demo to many partitions and workers without redesigning the application.

4. Dashboard Features

• **Live view** - auto-refreshes every 15 s (adjustable, pausable) using Streamlit fragments so only the live panel reloads; per-sensor KPI cards with change since the last reading; trend chart; geographic map with trail for position data; automatic fall-back to MongoDB if Kafka is unreachable.

• **History view** - presents *all* collected records with filters for time range, metric, sensor and API source; summary statistics; distribution plot; full sortable table.

• **Aggregation (additional points)** - Spark window aggregates from MongoDB, plus an interactive roll-up from 1 minute to 1 day with avg/min/max/std/count and a min-max band.

• **UI/UX (additional points)** - status badges for data mode, KPI cards, index-to-100 comparison when sensors have very different scales (e.g. BTC vs SOL), time-zone selector (default Asia/Manila), responsive layout, clear banners when demo data is shown.

• **Export (additional points)** - Excel (.xlsx) with formatted header, filters, summary sheet and native chart; PDF report with statistics, chart and latest rows; PNG and JPG chart images; CSV.

[Insert screenshot]
Live tab with KPI cards and trend chart

[Insert screenshot]
History tab with filters, summary and full table

[Insert screenshot]
Aggregation tab (Spark windows) and Export buttons

5. Significance of the Data Dashboard

A streaming dashboard turns a continuous flow of raw API responses into information people can act on while it is still relevant. The **live view** shortens the time between an event (a price move, an earthquake, a spike in PM2.5) and a decision. The **history view** preserves every record so trends, anomalies and seasonality can be examined later - something the source APIs themselves usually do not offer for free. **Aggregation** reduces noisy high-frequency data to meaningful statistics, and **export** lets results be shared with people who do not use the dashboard. Because the pipeline is source-agnostic, the same architecture can monitor air quality for public health, seismic activity for disaster preparedness, or market data for financial analysis - demonstrating why big-data tooling (Kafka, Spark, MongoDB) is used for high-volume, high-velocity data instead of a single script writing to a file.

6. Testing

Automated tests (`pytest`) cover: API adapters with mocked responses (schema, missing values, bad payloads, de-duplication); the MongoDB layer (idempotent upserts, filtered queries, aggregates); Spark parsing and window aggregation on a local Spark session; the producer's error isolation and Kafka keying; all five export formats (including empty data); and headless rendering of the dashboard with and without a database.

Add here the results of your own end-to-end run (screenshots of the Kafka producer log, Spark batch log and MongoDB collection counts).

[Insert screenshot]
Terminal: producer sending records / Spark batch logs / MongoDB document count

7. Limitations and Future Work

• Free APIs update at their own pace (e.g. CoinGecko about every 30-60 s), so consecutive 15 s polls can repeat a value.

• Spark writes each micro-batch to MongoDB through the driver; for much larger volumes the MongoDB Spark Connector should be used.

• Late data beyond the 2-minute watermark is stored in *readings* but excluded from window aggregates.

• Future work: anomaly detection / alerting, multi-broker Kafka, authentication on the dashboard.

8. Conclusion

The project delivers a working MongoDB-Kafka-Spark pipeline fed by real public APIs, with a 15-second live dashboard, a complete history, Spark-based aggregation and multi-format export. The modular design (adapter-based sources, schema-first messages, idempotent storage) makes it easy to extend and to explain.

References

• CoinGecko API documentation - https://docs.coingecko.com/reference/simple-price

• wheretheiss.at API - https://wheretheiss.at/w/developer

• Open-Meteo Air Quality API - https://open-meteo.com/en/docs/air-quality-api

• USGS Earthquake Catalog API - https://earthquake.usgs.gov/fdsnws/event/1/

• Apache Spark Structured Streaming + Kafka Integration Guide - https://spark.apache.org/docs/latest/streaming/

• Apache Kafka documentation - https://kafka.apache.org/documentation/

• MongoDB documentation - https://www.mongodb.com/docs/